

GUIA PRATICO

Evolution API + FastAPI

Instalacao do Docker, configuracao base, conexao do WhatsApp e testes de recebimento e envio.

Fluxo que sera validado

```
WhatsApp -> Evolution API (Docker) -> FastAPI webhook  
FastAPI webhook <- Evolution API <- WhatsApp
```

Escopo: ambiente local de desenvolvimento. Este guia nao cria OCR, IA, respostas automaticas ou regras avancadas de negocio.

Seguranca

Nunca envie chaves de API, senhas ou o conteudo completo do arquivo .env para outra pessoa. Use valores fortes fora do ambiente local.

1. Pre-requisitos

Este exemplo considera Ubuntu ou Debian e os projetos nos caminhos abaixo. Ajuste somente se sua instalacao estiver em outro diretorio.

Projeto FastAPI	/var/www/Pessoal/lapzap
Projeto Evolution	/var/www/Servidor/server_settings/evolution-api
Python	Python 3.12+ e ambiente virtual .venv
Portas	8000 para FastAPI; 8080 para Evolution API

Instalar Docker Engine

Siga preferencialmente a documentacao oficial do Docker para sua distribuicao. No Ubuntu, uma instalacao simples para desenvolvimento pode ser:

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
# encerre a sessao e entre novamente antes de usar docker sem sudo
docker --version
docker compose version
```

Se o comando docker retornar erro de permissao, entre novamente na sessao ou use sudo temporariamente. Nao use sudo para executar o FastAPI.

Instalar dependencias do FastAPI

```
cd /var/www/Pessoal/lapzap
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
```

2. Configurar a Evolution API

No diretorio da Evolution, crie ou revise o arquivo .env. Os valores abaixo formam a base local. Troque a chave de API por um segredo proprio.

```
cd /var/www/Servidor/server_settings/evolution-api
cp .env.example .env # somente se .env ainda nao existir
```

SERVER_PORT	8080
AUTHENTICATION_API_KEY	uma chave secreta, nao deixe vazia
DATABASE_PROVIDER	postgresql
DATABASE_CONNECTION_URI	postgresql://user:pass@postgres:5432/evolution_db?schemma=evolution_api
POSTGRES_DATABASE	evolution_db
POSTGRES_USERNAME	user
POSTGRES_PASSWORD	pass
CACHE_REDIS_URI	redis://evolution-redis:6379/6

Webhook global

Como FastAPI e Evolution estao na mesma maquina, a Evolution acessa o host por host.docker.internal. No .env da Evolution, configure:

```
WEBHOOK_GLOBAL_ENABLED=true
WEBHOOK_GLOBAL_URL=http://host.docker.internal:8000/api/v1/webhooks/whatsapp
WEBHOOK_GLOBAL_WEBHOOK_BY_EVENTS=false
WEBHOOK_EVENTS_MESSAGES_UPSERT=true
```

No docker-compose.yaml, o servico api precisa ter o gateway do host:

```
extra_hosts:
  - "host.docker.internal:host-gateway"
```

Tambem mantenha o alias postgres na rede evolution-net caso a DATABASE_CONNECTION_URI use postgres como host.

3. Subir os servicos

Inicie a Evolution API e seus servicos auxiliares. O primeiro inicio baixa as imagens e pode demorar alguns minutos.

```
cd /var/www/Servidor/server_settings/evolution-api
docker network inspect dokploy-network >/dev/null 2>&1 || docker network create dokploy-network
docker compose up -d
docker compose ps
curl http://localhost:8080/
```

A ultima chamada deve responder com status 200 e informar que a Evolution API esta funcionando.

Subir o webhook FastAPI

```
cd /var/www/Pessoal/lapzap
source .venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Mantenha esse terminal aberto. Cada evento recebido aparecera nele como POST /api/v1/webhooks/whatsapp com HTTP 200.

Configuracao local do LapZap

```
# /var/www/Pessoal/lapzap/.env
EVOLUTION_API_URL=http://localhost:8080
EVOLUTION_API_KEY=<mesma chave de AUTHENTICATION_API_KEY>
EVOLUTION_INSTANCE_NAME=lapzap-dev
```

A chave ainda nao e usada pelo webhook de entrada, mas sera necessaria quando o FastAPI fizer chamadas para a Evolution no futuro.

4. Conectar o WhatsApp

Defina as variáveis apenas para o terminal atual. Elas evitam repetir a chave em cada chamada e não mostram o segredo na tela.

```
cd /var/www/Servidor/server_settings/evolution-api
export EVO_API_KEY="$(sed -n 's/^AUTHENTICATION_API_KEY=//p' .env | head -n 1)"
export INSTANCE_NAME="lapzap-dev"
[ -n "$EVO_API_KEY" ] && echo "Chave carregada" || echo "Chave não encontrada"
```

Criar a instância e solicitar QR Code

```
curl -sS -X POST http://localhost:8080/instance/create \
-H "apikey: $EVO_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "instanceName": "'"$INSTANCE_NAME"'",
  "integration": "WHATSAPP-BAILEYS",
  "qrcode": true
}'
```

Para salvar o QR Code como imagem e abrir no computador:

```
curl -sS -H "apikey: $EVO_API_KEY" \
"http://localhost:8080/instance/connect/$INSTANCE_NAME" \
| jq -r '.base64 // .qrcode.base64' \
| sed 's#^data:image/png;base64,##' \
| base64 -d > /tmp/lapzap-qr.png
xdg-open /tmp/lapzap-qr.png
```

No celular: WhatsApp -> Configurações -> Dispositivos conectados -> Conectar dispositivo.
Escaneie o QR Code.

```
curl -sS -H "apikey: $EVO_API_KEY" \
"http://localhost:8080/instance/connectionState/$INSTANCE_NAME"
```

O teste está pronto quando a resposta tiver state igual a open.

5. Testar recebimento e envio

Recebimento real

Com a instancia aberta e o FastAPI em execucao, envie uma mensagem de outro celular para o numero conectado na Evolution. O terminal FastAPI deve registrar HTTP 200. O fluxo real e:

```
WhatsApp de teste -> Evolution API -> POST /api/v1/webhooks/whatsapp -> log FastAPI
```

Envio manual pela Evolution

Este comando envia uma mensagem da conta conectada para o telefone de teste. Use o numero somente com codigo do pais e DDD, sem +, espacos ou hifens.

```
curl -sS -X POST \
  "http://localhost:8080/message/sendText/$INSTANCE_NAME" \
  -H "apikey: $EVO_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "number": "5585921674573",
    "text": "Ola! Mensagem de teste enviada pela Evolution API."
  }'
```

A resposta bem-sucedida usa HTTP 201. Responder por esse outro telefone testa a entrada real no webhook.

Teste manual do webhook

```
curl -X POST http://localhost:8000/api/v1/webhooks/whatsapp \
  -H "Content-Type: application/json" \
  -d '{
    "event": "messages.upsert",
    "instance": "lapzap-dev",
    "data": {
      "key": {"remoteJid": "5585921674573@s.whatsapp.net", "id": "teste-001"},
      "message": {"conversation": "Teste do webhook"}
    }
  }'
```

Resposta esperada: {"status":"ok"}.

6. Diagnostico rapido

401 Unauthorized	Carregue EVO_API_KEY a partir do .env e envie o header apikey.
Cannot GET /instance/connectionState/	INSTANCE_NAME esta vazio. Execute export INSTANCE_NAME=lapzap-dev.
Webhook retorna 500	Confira o terminal FastAPI. Payloads sem objeto em data devem agora ser ignorados com 200.
Evolution nao chama FastAPI	Confirme FastAPI em 0.0.0.0:8000 e WEBHOOK_GLOBAL_URL usando host.docker.internal.
Erro Redis disconnected	Use CACHE_REDIS_URI=redis://evolution-redis:6379/6 e recrie o container api.
Permissao no Docker	Entre novamente apos usermod -aG docker \$USER ou use sudo docker compose.

Comandos de observacao

```
cd /var/www/Servidor/server_settings/evolution-api
docker compose ps
docker compose logs -f api
docker compose logs -f redis

# em outro terminal
cd /var/www/Pessoal/lapzap
source .venv/bin/activate
pytest -q
```

Desconectar a conta

No celular, abra WhatsApp -> Configuracoes -> Dispositivos conectados, escolha a sessao vinculada a Evolution e toque em Desconectar. A instancia permanece criada, mas deixa de receber mensagens.

Proximos passos

Com o fluxo validado, os proximos incrementos podem ser: persistir mensagens recebidas, autorizar numeros cadastrados, tratar imagens e depois integrar OCR ou IA. Mantenha cada etapa separada para facilitar testes.